

MODULE 11

A09: Logging & Alerting Failures

50-Minute Integrated Practical | OWASP Top 10:2025 | BBC Academy / JBI Training

Tools: ShopCo (vulnerable_shop.py & vulnerable_shop_fixed.py) • OWASP Juice Shop • ZAP Proxy • Flask Security Supplement

Note: "Monitoring" renamed to "Alerting" in OWASP 2025 — logs with no alerts are silent failures.

COURSE POSITION: Module 11 is the detection strand. Every vulnerability you have attacked since Module 5 produced an event that should have been logged. A09 asks: was it? Cross-references: Module 5 (A01+A07 — login events, IDOR attempts); Module 6 (A05 — SQLi/XSS/command injection payloads); Module 7 (SSDLC — logging as an operational control, ShopCo); Module 9 (A04+A08 — JWT tampering, basket manipulation, exception correlation IDs); Module 10 (A06+A10 — fail-closed design produces log entries; Activity 1 Expert challenge designed exactly this handler).

Activity	Focus	Tool	Time
Activity 1	Attack Blind — the cost of A09 with ShopCo	ShopCo vulnerable_shop.py	12 min
Activity 2	Enable Structured Logging — repeat, compare	ShopCo vulnerable_shop_fixed.py	10 min
Activity 3	Juice Shop Logging Audit — what it logs, what it misses	Juice Shop • ZAP	13 min
Activity 4	Alert Threshold Design & UK GDPR Constraints	Group exercise	10 min
Activity 5	Defence Mapping & Course-Wide Connection	Group discussion	5 min

ACTIVITY 1 — Attack Blind: What A09 Costs You ⌚ 12 min

A09 Logging & Alerting Failures • ShopCo vulnerable_shop.py • Terminal

WHY IT MATTERS: OWASP A09 renamed 'Monitoring' to 'Alerting' in 2025 to make the point explicit: logs that produce no alerts are silent failures. An attacker who succeeds against a system with no security logging leaves no forensic trail. The SOC sees nothing. The incident timeline cannot be reconstructed. Regulatory obligations (UK GDPR Article 33: 72-hour breach notification) cannot be met without logs. This activity makes that gap visceral by running real attacks and observing what the server records — which is nothing.

Step-by-step

1. Open a terminal (Ctrl+Alt+T). Navigate to the ShopCo repository: `cd ~/shopco-security-lab`
2. Start the vulnerable (unlogged) application: `python3 vulnerable_shop.py`
3. Open a second terminal side-by-side. This will show you the server output in real time as attacks occur.
4. In Firefox, navigate to the ShopCo login page: `http://localhost:5000/login`
5. Run the SQL injection bypass from Module 6. Username: `admin' --`
6. Observe both terminals. In the server terminal you should see only the default Flask access log line, e.g.:
`127.0.0.1 - - [date] "POST /login HTTP/1.1" 302 -`

- Now run 10 rapid login attempts — use the SQLi payload each time. Use the browser's back button and repeat quickly. Watch the server terminal.
- In ZAP, verify the full payload is visible in History. Now answer from the server log alone — circle what you CAN and CANNOT tell:

Question	From the server log	From ZAP History
Who was targeted?	Cannot tell	Username 'admin' visible
What payload was used?	Cannot tell	Full SQLi payload visible
Did authentication succeed?	Cannot tell — 302 for both	Success vs failure visible in response
How many attempts were made?	Cannot tell	Count every POST /login
Where did the attack come from?	IP only — no context	IP + User-Agent + headers
Was this a credential-stuffing attack?	Cannot tell	Repeated identical payload visible

A09 FAILURE DEFINED: An attack succeeded. Ten attempts were made. Authentication was bypassed using a ShopCo Flaw 4 payload (SQL injection, Module 6). Not one of these facts appears in the server log. A SOC analyst reviewing this system would see nothing unusual. The UK GDPR 72-hour breach notification clock has already started — and you have no evidence to file with the ICO.

EXPLORER	Record the six answers in the table above. Write one sentence explaining why a 302 HTTP response code is insufficient to distinguish a legitimate login from an SQLi bypass.
PRACTITIONER	In the server terminal, count the total number of POST /login log lines. Compare this to the number of ZAP History entries. Now explain: if the 302 response code hides success/failure, what single piece of information in the structured log (Activity 2) would make all 10 attempts immediately visible as an attack pattern?
EXPERT	Write a SIEM correlation rule (in plain English, not a specific SIEM query language) that would alert on this exact attack pattern. Specify: event type, threshold count, time window, grouping field (per-IP or per-username), and the alert severity. Connect this to the rate-limiting design from Module 5 (Flask-Limiter) — how do rate-limiting and alerting complement each other?

ACTIVITY 2 — Enable Structured Logging: Same Attack, Different Visibility

10 min

A09 Logging & Alerting Failures · ShopCo vulnerable_shop_fixed.py · Flask Security Supplement

WHAT CHANGES: The Flask Security Supplement's `log_event()` function (Module 11 section) is already integrated into `vulnerable_shop_fixed.py`. It emits structured JSON security events to a dedicated 'security' logger. You will repeat the identical attacks from Activity 1 and observe what a defender now sees. Note: `vulnerable_shop_fixed.py` runs on port 5001 to allow both versions to run simultaneously for direct comparison.

Step-by-step

- Keep `vulnerable_shop.py` running on port 5000. In the second terminal, start the fixed app: `python3 vulnerable_shop_fixed.py`
- Navigate to the fixed app login page: `http://localhost:5001/login`
- Repeat the SQL injection login bypass — same payload as Activity 1: `admin' --`
- Observe the server terminal. You should see a structured JSON event similar to:

```
{ "timestamp": "2026-03-20T10:23:41Z",
```

```
"event": "LOGIN_FAILURE",
"username": "admin'--",
"source_ip": "127.0.0.1",
"success": false,
"detail": "invalid_credentials"}
```

13. Repeat the attack 10 times rapidly. Watch the terminal. Notice: (a) every attempt produces a separate structured event; (b) the username field contains the raw SQLi payload; (c) a SIEM rule can now trivially count LOGIN_FAILURE events from the same source_ip within a time window.
14. Now attempt an IDOR attack — access another user's profile by changing the user_id URL parameter (Module 5 Flaw 7 / ShopCo profile route). What event does the fixed app emit? Is it different from the login events?
15. Compare the structured log to ZAP History. Can the log alone reconstruct the attack timeline without needing ZAP? What is still missing from the structured log that ZAP captures?

The log_event() function — understand what it does

The Flask Security Supplement defines this function. Locate it in vulnerable_shop_fixed.py and read it:

```
def log_event(event_type, username, ip, success, detail=''):
    security_log.warning(json.dumps({
        'timestamp': datetime.utcnow().isoformat(),
        'event': event_type,
        'username': username,    # username attempted — NOT the password
        'source_ip': ip,
        'success': success,
        'detail': detail
    })))
```

CRITICAL DESIGN NOTE: Notice: username is logged, password is not. This is intentional and correct. A log file containing passwords — even hashed ones — is a credential database. If an attacker exfiltrates your logs, they gain offline attack material. The log records the fact that a login attempt occurred for a given identity. It never records what credential was submitted. This is a UK GDPR / DPA 2018 requirement — see Activity 4.

EXPLORER	Repeat the SQLi login bypass and observe the structured JSON event. Write down all six fields and explain in one sentence why 'success: false' is more valuable for a SOC analyst than the HTTP 302 status code alone.
PRACTITIONER	Trigger three different event types: LOGIN_FAILURE, LOGIN_SUCCESS, and an IDOR attempt on the profile route. Record the full JSON for each. Identify: which fields are common to all three events, and which fields differ? What does this consistency enable for a SIEM correlation rule?
EXPERT	Extend the log_event() function to add a 'correlation_id' field — a UUID generated per request and shared with both the server log and the HTTP response as an X-Correlation-ID header. Explain how this connects Module 10 Activity 1's Expert challenge (error handlers) with Module 11's logging requirement: a single correlation_id ties an A10 exception entry to its A09 security event.

ACTIVITY 3 — Juice Shop Logging Audit: What Does It Log? What Should It?

🕒 13 min

A09 Logging & Alerting Failures · OWASP Juice Shop · ZAP Proxy

WHY IT MATTERS: Juice Shop is a deliberately vulnerable application — it is also deliberately under-logged. Several of the attacks you performed in Modules 5–10 succeeded without producing a single log entry that would reveal the attack. This activity turns you into the detective: perform each attack, check what Juice Shop logs in its terminal output, and identify the gap between what was logged and what a production application should have recorded.

Step-by-step

- Start Juice Shop if not already running. Open a terminal alongside it so you can observe console output in real time: `cd ~/juice-shop && npm start`
- Perform the following attacks in sequence, watching the Juice Shop terminal after each one. Use ZAP to confirm the attack payload was sent. Record what — if anything — appears in the Juice Shop terminal.

Module Ref	Attack Performed	Juice Shop Terminal Output	What Should Have Been Logged
Module 5 (A07)	Login with admin@juice-sh.op and wrong password 5× in 30 seconds	HTTP 401 lines — no structured event	LOGIN_FAILURE with user_id, source_ip, attempt_count, timestamp per attempt. Alert threshold: 5 failures / 60s.
Module 5 (A07)	Successful login after 5 failures	HTTP 200 — no correlation to prior failures	LOGIN_SUCCESS with user_id, source_ip, timestamp. SIEM rule: SUCCESS after 5+ FAILURE from same IP = suspicious.
Module 6 (A05)	SQLi payload in product search: ' OR 1=1--	HTTP 200 — query executes, no error logged	INJECTION_ATTEMPT: event, endpoint, sanitised payload excerpt (no raw SQLi), source_ip, user_id.
Module 9 (A08)	PUT /api/BasketItems with modified BasketId (another user's basket)	HTTP 200 — operation completes silently	AUTHORISATION_FAILURE: attempted resource, requesting user_id, resource owner_id, endpoint, timestamp.
Module 9 (A08)	JWT token with forged role=admin claim	HTTP 200 or 401 — no signature event	TOKEN_VALIDATION_FAILURE: reason (signature_mismatch / role_escalation), source_ip, token_subject.
Module 10 (A06)	GET /#/administration as non-admin user	HTTP 200 — admin data returned, nothing logged	PRIVILEGE_ESCALATION_ATTEMPT: user_id, user_role, resource_accessed, required_role, timestamp.

EXPLORER	Complete the first two rows (login failure and success). Describe in one sentence what a SIEM analyst could determine from your 'What Should Have Been Logged' entries that they cannot determine from Juice Shop's actual output.
PRACTITIONER	Complete all six rows. Identify which of the six attack events is the most dangerous to miss — and justify your answer in terms of: (a) time available to respond, (b) data at risk, and (c) regulatory obligation under UK GDPR Article 33.
EXPERT	Write a Python snippet using the log_event() pattern from Activity 2 for the JWT validation failure event. The entry must include: event type, token subject (sub claim), claimed role, source_ip, reason (signature_mismatch / invalid_algorithm / role_escalation), and a correlation_id. Explain how this log entry would feed a SIEM rule to detect privilege escalation patterns across multiple endpoints.

ACTIVITY 4 — Alert Threshold Design & UK GDPR Constraints 10 min

A09 · Group exercise · Pen & paper — no tools required

THE TWO HALVES OF A09: A09 has equal weight on both sides: what you must log (to enable detection and meet regulatory obligations) and what you must never log (to avoid creating a GDPR liability or a credential database). This activity addresses both. Part A: design alert thresholds. Part B: identify what must be scrubbed from a log before it is stored.

Part A — Alert Threshold Design (5 min): Fill in the threshold column

For each event type below, write: (1) the numeric threshold that should trigger an alert, (2) the time window, and (3) the alert severity. Compare with your partner — there are no single correct answers, but you must be able to justify your choice.

Event Type	Grouping Field	Threshold	Time Window & Severity	Why This Threshold?
LOGIN_FAILURE	Per source_ip	5	60s → HIGH	Brute force / credential stuffing (Module 5 rate-limit design)
LOGIN_FAILURE	Per username	10	3600s → MEDIUM	Account enumeration across distributed IPs
LOGIN_SUCCESS after 5+ LOGIN_FAILURE	Per source_ip + username	1	300s → HIGH	Credential stuffing success — Module 5 A07
AUTHORISATION_FAILURE	Per user_id	3	300s → HIGH	IDOR probing — Module 5 A01
INJECTION_ATTEMPT	Per source_ip	1	N/A → CRITICAL	Single SQLi/XSS attempt is always alertable — Module 6
PRIVILEGE_ESCALATION_ATTEMPT	Per user_id	1	N/A → CRITICAL	Any admin panel access by non-admin — Module 10 A06
TOKEN_VALIDATION_FAILURE	Per source_ip	3	600s → HIGH	JWT forgery attempts — Module 9 A08
5XX_ERROR	Global	50	60s → MEDIUM	Application error spike — Module 10 A10

Part B — UK GDPR / DPA 2018: What to Scrub Before Storing Logs (5 min)

The log entries below contain real data from a test system. Identify every field that must be scrubbed, truncated, or replaced before the log is stored persistently. Write the corrected version in the right column.

Raw Log Entry (as written by a developer)

```
{"event":"LOGIN_FAILURE","username":"alice","password":"Passw0rd!","ip":"203.0.113.47"}
```

```
{"event":"PAYMENT","card":"4111111111111111","amount":49.99,"user":"alice@bbc.co.uk"}
```

```
{"event":"SESSION_CREATE","token":"eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhZG1pbWJ9.xyz","user_id":"usr_001"}
```

```
{"event":"ERROR_500","traceback":"File /home/ubuntu/app.py line 247...","user_ip":"10.0.0.42"}
```

EXPLORER

Complete Part A rows 1–2 and Part B rows 1–2. Explain in one sentence why logging a password — even a hashed one — creates a security liability rather than a security benefit.

PRACTITIONER

Complete all of Part A and Part B. For Part B row 3 (session token): explain why storing a full JWT in a log file is equivalent to storing a plaintext password, and what an attacker who gained read access to the log could do with it.

EXPERT

Part A: your INJECTION_ATTEMPT threshold says 'alert on first occurrence'. Write the counter-argument: why might a production environment tolerate 1–2 false positives from legitimate edge cases? Then explain what additional context (e.g. whether the user is authenticated, whether the payload matches known OWASP test patterns) a SIEM rule could use to reduce false positives without raising the threshold.

ACTIVITY 5 — Defence Mapping: Connecting Logging to Every Module ⌚ 5 min

Group — 5 minutes · map what to log for each attack seen across the course

Module	Attack Performed	OWASP Category	Event to Log	Alert Threshold
Module 5	SQLi login bypass — ShopCo Flaw 4	A05 Injection	LOGIN_FAILURE (username, ip, success=false)	5 per IP per 60s → HIGH
Module 5	IDOR: user_id from URL — ShopCo Flaw 7	A01 Broken Access Control	AUTHORISATION_FAILURE (user_id, resource, owner_id)	3 per user per 5min → HIGH
Module 5	Admin panel accessed without role — Juice Shop	A06 Insecure Design	PRIVILEGE_ESCALATION_ATTEMPT (user_id, role, endpoint)	1 occurrence → CRITICAL
Module 6	XSS payload submitted in search or feedback	A05 Injection	INJECTION_ATTEMPT (endpoint, payload_type, user_id)	1 occurrence → CRITICAL
Module 7	pip-audit CVE in production dependency	A03 Supply Chain	DEPENDENCY_CVE_DETECTED (package, version, cvss, cve_id)	Any CVSS ≥ 7.0 → HIGH
Module 9	JWT claim tampered or alg:none used	A08 Integrity Failure	TOKEN_VALIDATION_FAILURE (reason, sub, source_ip)	3 per IP per 10min → HIGH
Module 9	Basket price/quantity tampered via ZAP	A08 Integrity Failure	INTEGRITY_VIOLATION (endpoint, field, user_id, value)	1 occurrence → HIGH
Module 10	Exception reveals stack trace in response	A10 Exceptional Conditions	ERROR_DISCLOSURE_ATTEMPT (endpoint, correlation_id, status)	10 per IP per 60s → MEDIUM

KEY TAKEAWAY: Every attack in this course — from Module 5 to Module 10 — produced a loggable event. A09 is not a standalone module: it is the retrospective thread that runs through every other OWASP category. The question is not 'should we log this?' — it is 'what is the minimum structured event that makes this attack detectable, actionable, and legally defensible within 72 hours of discovery?'

Further Reading & Resources

- OWASP Top 10:2025 — A09 Logging & Alerting Failures: owasp.org/Top10/A09_2021-Security_Logging_and_Monitoring_Failures/
- OWASP Cheat Sheet: Logging: cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html
- OWASP Juice Shop — Logging CTF challenges: [pwning.owasp-juice.shop/part2/security-misconfiguration\(logging misconfiguration challenges\)](https://pwning.owasp-juice.shop/part2/security-misconfiguration(logging%20misconfiguration%20challenges))
- UK ICO: logging and breach notification under UK GDPR Article 33: ico.org.uk/for-organisations/guide-to-data-protection/guide-to-the-general-data-protection-regulation-gdpr/personal-data-breaches/
- NIST SP 800-92 — Guide to Computer Security Log Management: csrc.nist.gov/publications/detail/sp/800-92/final
- Flask Security Supplement — Module 11 section: `log_event()` and structured logging pattern (in your lab VM)
- MITRE ATT&CK — T1562.002 (Disable Windows Event Logging) and T1070 (Indicator Removal on Host): attack.mitre.org